Departamento de Informática, Univ. de Valencia

Sistemas Inteligentes

Juan Domingo
Juan.Domingo@uv.es

Septiembre de 2026

Tema 3:

Teoría de juegos

Índice del tema 3: comienzo de la sección 1

1 Formalismo

2 Juegos posicionales: árboles de juego
2.1 Modelos de árboles
2.2 Algoritmo Minimax
2.3 Algoritmo $\alpha - \beta$
2.4 Arboles para juegos parcialmente aleatorios

3 Juegos no posicionales
3.1 Juegos de suma nula
3.2 Juegos de suma no nula

Definición

Teoría de juegos

Es la teoría matemática que modeliza las interacciones entre dos o más agentes, teniendo en cuenta su ambiente, en el que puede haber o no elementos aleatorios.
La teoría asume que

- Los agentes son **racionales**, en el sentido explicado en el tema anterior. En particular, actuarán sólo y siempre para cumplir sus metas programadas y asumirán que los otros agentes también lo harán.
- El grado de cumplimiento de la(s) meta(s) es cuantificable, con una función numérica distinta para cada agente.
- La función que cada agente trata de optimizar es la propia, sin importarle el valor que tomen las funciones de los demás agentes (aunque sí sus decisiones). La excepción podrían ser los juegos cooperativos, pero incluso en este caso se suele dar una sóla función objetivo.

Contexto

Hay que aclarar que, pese a su nombre, esta teoría no se restringe sólo a juegos; en general, sirve para la toma de decisiones óptimas y/o la resolución de conflictos. Es, en realidad, un caso particular de la teoría de la decisión, cuyos otros casos son:

		Número de objetivos	
		1	≥ 2
Número de agentes	1	Investigación operativa	Decisión multicriterio
	≥ 2	Juegos cooperativos	Juegos no cooperativos

Nos centraremos sólo en los casos más sencillos de juegos **no cooperativos**, es decir, aquellos en los que cada agente tiene un objetivo distinto (normalmente, ganar él mismo).

Clasificación (I)

Los juegos no cooperativos se dividen en

- Posicionales: cada agente toma un decisión, y la ejecuta (esto se llama **jugada**) alterando el ambiente, un agente tras otro, en orden prefijado. Gana el agente que primero consigue que el ambiente alcance algún estado que le es favorable, al que se llama **estado meta** o estado objetivo de ese agente (puede haber más de uno).
- No posicionales: todos los agentes toman una decisión a la vez sin conocer las que están tomando sus adversarios. El resultado de todas esas decisiones determina el beneficio/perjuicio de cada agente (y por tanto, quién gana el juego).

Clasificación (II)

A su vez, los juegos posicionales se dividen en

- Deterministas: cualquier agente puede tomar cualquier decisión que las reglas le permitan en cada jugada.
- No deterministas: hay un elemento de azar que restringe las decisiones que un agente puede tomar en una jugada particular.

Igualmente, los juegos no posicionales se dividen en

- Matriciales, también llamados de suma nula, si la suma de las ganancias y pérdidas de todos los agentes tras el juego es 0 (o alguna constante).
- Bimatriciales, también llamados de suma no nula, si dicha suma no es 0 (o no es constante).

Veremos formalismos para estos cuatro tipos, pero restringiéndonos sólo a juegos de **dos jugadores**. Podemos adelantar algunos ejemplos de ellos:

No posicional	matricial	Piedra/papel/tijera
	bimatricial	El dilema del prisionero
Posicional	determinista	Ajedrez
	no determinista	Backgammon

Índice del tema 3: comienzo de la sección 2

1 Formalismo

2 Juegos posicionales: árboles de juego
2.1 Modelos de árboles
2.2 Algoritmo Minimax
2.3 Algoritmo $\alpha - \beta$
2.4 Arboles para juegos parcialmente aleatorios

3 Juegos no posicionales
3.1 Juegos de suma nula
3.2 Juegos de suma no nula

Modelo de un árbol de juego (I)

Hasta ahora la evolución de un sistema con estados (es decir, la elección del nodo al que nos movemos desde el actual, de entre sus descendientes) se suponía efectuada por un sólo agente. Sin embargo, en juegos de dos o más jugadores varios agentes eligen alternativamente.

Es posible considerar como estados sucesores a cada una de las jugadas posibles del contrario, seguidas de cada una de nuestras posibles réplicas, pero esto es complejo y no muy natural.

En su lugar, lo que haremos será considerar un árbol especial llamado **árbol de juego** donde cada nodo es un estado del "tablero" y donde los arcos que parten de un nodo representan las acciones posibles para el jugador digamos A, si el nivel de profundidad es par, y las acciones posibles para el jugador B si el nivel es impar.

Modelo de un árbol de juego (II)

Además de eso cada jugador intenta alcanzar una meta distinta (y opuesta) con lo que el algoritmo de búsqueda no se puede basar en buscar un camino al nodo meta.

Veremos métodos que operan sobre estos árboles alternos. Todos son perfectos (es decir, llevan a la victoria o al empate) cuando se aplican al árbol completo, sin límite de profundidad, pero esto es computacionalmente intratable (tanto espacial como temporalmente) en cualquier juego mínimamente complejo, por lo cual el árbol debe podarse a un nivel de profundidad prefijado, que aquí se llama **horizonte de la búsqueda**.

Algoritmo Minimax (I)

Consiste en explorar el árbol de juego de modo exhaustivo hasta un nivel de profundidad prefijado. Si el nivel fuera la profundidad del árbol (lo que es impracticable) el método daría la mejor jugada para el jugador que va a mover desde la raíz. Si no, se obtiene una solución subóptima, que incluso podría revelarse como perjudicial si se incrementase la profundidad. A esto se le llama **efecto horizonte**

Este algoritmo no se usa en la práctica, pero es la base del siguiente (el $\alpha - \beta$), por lo que debe explicarse.

El objetivo es elegir la mejor acción a tomar en el nodo raíz. Nos referiremos al jugador que mueve desde el nodo raíz como 'nosotros'. Tenemos una función de evaluación de los nodos que los ordena por preferencia. Toma un valor máximo en los nodos terminales en los que ganamos nosotros y mínimo en los que perdemos. En los demás será mayor cuanto mayor sea la ventaja que nos da ese estado sobre nuestro oponente.

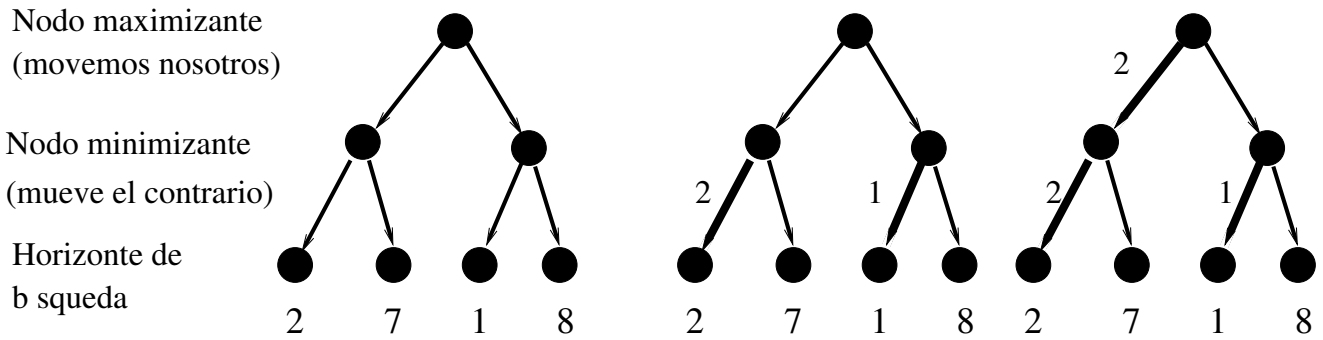
Algoritmo Minimax (II)

ATENCION: la función de evaluación es una heurística (el programador la construye y programa basándose en sus conocimientos o creencias acerca del juego) y refleja la ventaja que cree que le da **cada** estado (nodo) considerado aisladamente. No debe incluir consideraciones sobre cuántas jugadas creo que faltan para ganar, o en general, sobre el desarrollo futuro del juego, que se verá condicionado por las acciones de nuestro oponente, y que irán siendo tomadas en cuenta por **el propio algoritmo minimax a medida que éstas se produzcan**.

Este algoritmo no juega a ganar, sino a no perder. Puesto que no se conoce lo que hará el contrario, se elige el movimiento tal que haga lo que haga éste, se reduzcan nuestras pérdidas.

Algoritmo Minimax (III)

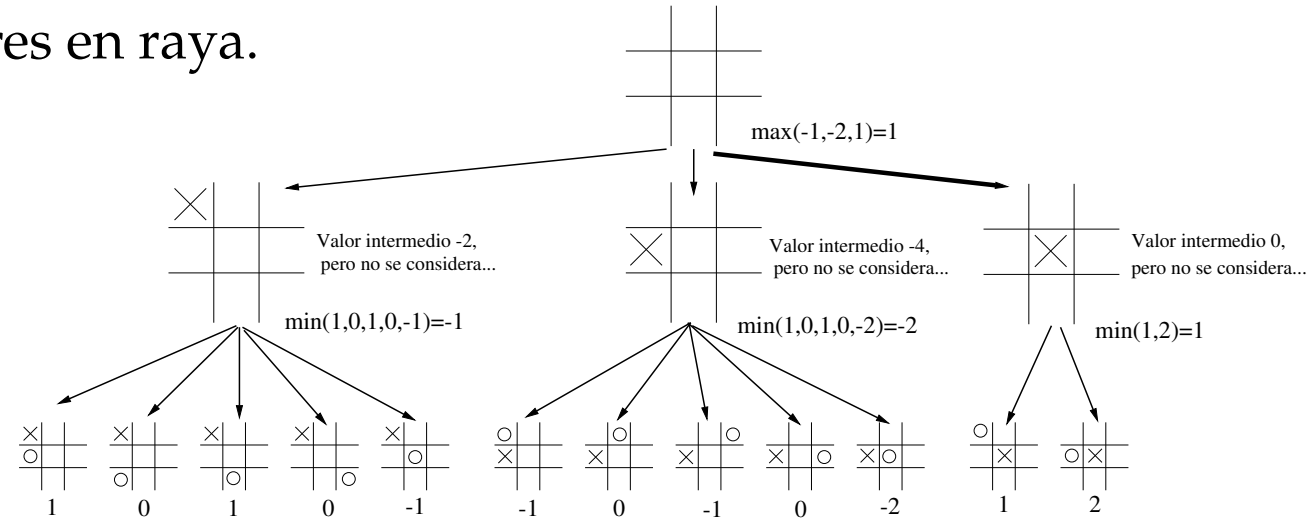
Nuestra posición actual constituye la raíz del árbol. La raíz es siempre lo que se llama **nodo maximizante** (intenta buscar el descendiente con el máximo valor de la heurística). Los nodos del siguiente nivel serán **nodos minimizantes**, porque el contrario habrá jugado para obtener su mejor resultado, que será el peor para nosotros, y queremos que esa situación nos resulte lo menos mala posible. Los nodos del siguiente nivel volverán a ser maximizantes, y así hasta llegar al horizonte de búsqueda.



Descendiendo hasta la profundidad del horizonte de búsqueda, (sea p) se elegirá la rama que de menor valor (si el nivel $p-1$ era minimizante) o mayor (si era maximizante) y este valor se propaga al nivel $p-1$. Se itera hasta llegar a la raíz. (Comentar qué pasa si el contrario no hace lo mejor para sí mismo).

Algoritmo Minimax (IV)

Ejemplo: tres en raya.



El horizonte se ha elegido en el segundo nivel, y la función de evaluación elegida ha sido la suma de líneas en las que podemos hacer tres en raya, menos la suma de líneas en las que el contrario puede hacer tres en raya. El árbol se ha simplificado utilizando simetrías, de forma que se han suprimido todos los movimientos simétricos de alguno existente, ya que darían el mismo resultado. Al aplicar el método minimax llegamos rápidamente a elegir el movimiento de tirar al centro, pues es el movimiento que garantiza un mayor desequilibrio a nuestro favor en el número de líneas que cada jugador tiene disponibles para hacer tres en raya.

Algoritmo Minimax (V)

Implemented as a recursive function, Minimax(node,depth,maximPlayer) where
- node: the node to start search
- depth: the maximal depth we want to arrive from node
- maximPlayer: TRUE if we are the maximizing player, FALSE otherwise

Variables and function:
- TerminalNodes: the set of nodes in which one player wins, or nobody can move.
- h(n): The heuristics for each node n.

```
function Minimax(node,depth,maximPlayer)
    if (depth == 0) or node ∈ TerminalNodes
        return h(node)
    if maximPlayer == TRUE
        bestValue ⟵ −∞
        for each child of node
            v ⟵ Minimax(child,depth-1,FALSE)
            bestValue ⟵ max(bestValue,v)
        return bestValue
    else          // minimizing player
        bestValue ⟵ +∞
        for each child of node
            v ⟵ Minimax(child,depth-1,TRUE)
            bestValue ⟵ min(bestValue,v)
        return bestValue
```

The initial call is

Minimax(startNode,depth,TRUE)

with any value for depth ≥ 1.

Algoritmo $\alpha - \beta$ (I)

Es una mejora del minimax, y aunque fue informalmente descrito en la conferencia de Darmouth no se publica un análisis preciso hasta 1963.
$\alpha - \beta$ obtiene **el mismo** resultado que el minimax (el nodo óptimo de entre todos los accesibles a profundidad menor que un límite, p), pero lo hace más eficientemente porque no explora todo el árbol hasta la profundidad p.
Cuando va a acceder a alguna rama en la que **no** puede encontrarse el óptimo, no lo hace (esto se llama **poda** del árbol).

¿Cómo sabe $\alpha - \beta$ que se puede podar una rama?.
Básicamente, deja de evaluar un movimiento en cuanto puede estar seguro de que el movimiento no puede ser mejor que cualquier otro previamente examinado.

Para eso se mantienen dos valores:

α: peor puntuación que el jugador maximizante está ahora seguro de poder alcanzar (máximo hasta ahora).

β: mejor puntuación que el jugador minimizante está ahora seguro de poder alcanzar (mínimo hasta ahora).

α se inicializa a $-\infty$ y va creciendo al explorar.

β se inicializa a ∞ y va decreciendo al explorar.

Cuando se cruzan (o sea, $\beta \leq \alpha$) se puede dejar de explorar. ¿Por qué?
Porque el jugador maximizante sabe que en las ramas que quedan por explorar no le puede pasar nada peor.

Algoritmo $\alpha - \beta$ (II)

Implemented as a recursive function, AlphBe(node,depth,maximPlayer,$\alpha\triangleleft$,$\beta\triangleleft$)
where node, depth, maximPlayer, TerminalNodes and h(n) are as in minimax.

```
function AlphBe(node,depth,maximPlayer,α◁,β◁)
   if (depth == 0) or node ∈ TerminalNodes
      return h(node)
   if maximPlayer == TRUE
      bestValue ⟵ −∞
      for each child of node
         v ⟵ AlphBe(child,depth-1,FALSE,α,β)
         bestValue ⟵ max(bestValue,v)
         α ⟵ max(α,v)                          ◁
         if (β≤α)                              ◁
            return bestValue                   ◁
      return bestValue
   else          // minimizing player
      bestValue ⟵ +∞
      for each child of node
         v ⟵ AlphBe(child,depth-1,TRUE,α,β)
         bestValue ⟵ min(bestValue,v)
         β ⟵ min(β,v)                          ◁
         if (β≤α)                              ◁
            return bestValue                   ◁
      return bestValue
```

The initial call is

minimax(startNode,depth,TRUE,$-\infty$,∞)

with any value for depth ≥ 1.

The elements marked with $\triangleleft$ are **additions** with respect to minimax.

The points in which we make a return **inside** a for loop (the returns with $\triangleleft$) are the points of **tree prunning**.

Notice that the prunning condition

$\beta \leq \alpha$

is **the same** for both players.

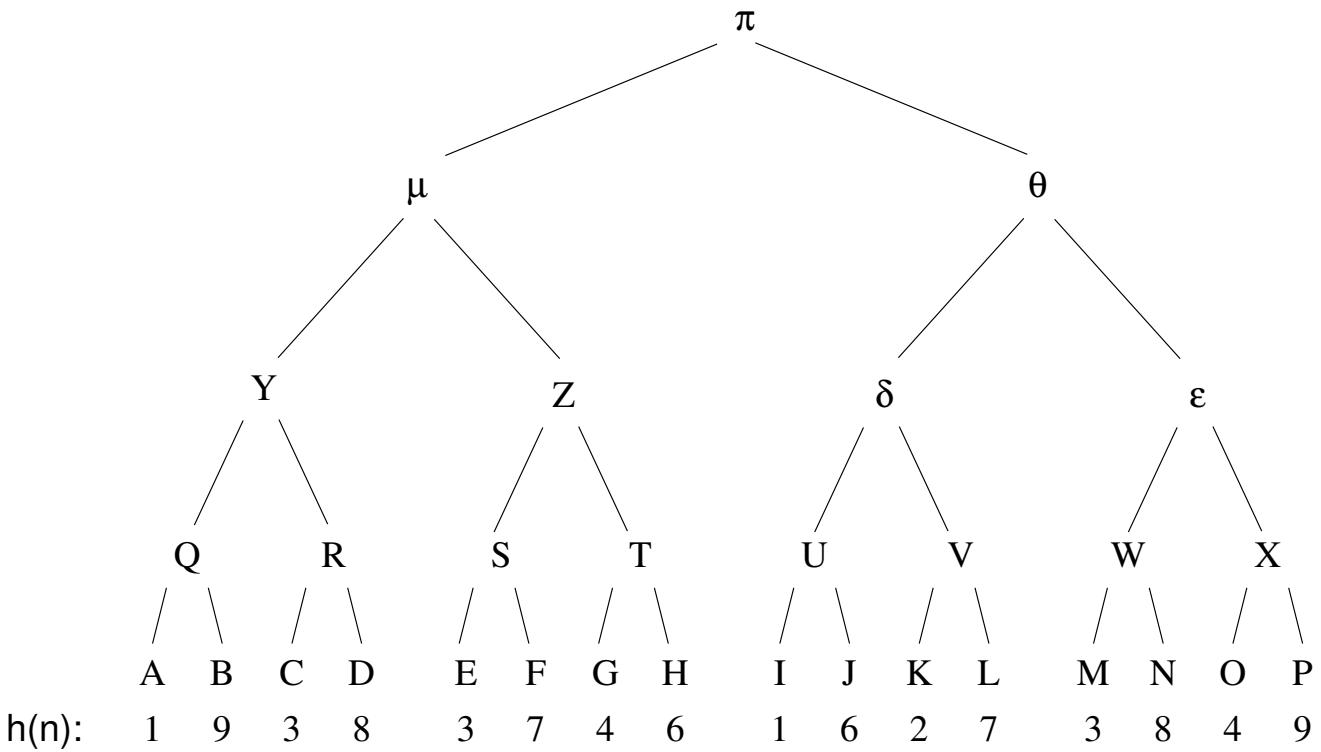
Algoritmo $\alpha - \beta$ (III)

Dos consideraciones importantes sobre el algoritmo $\alpha - \beta$:

- Como ya se dijo, el uso de $\alpha - \beta$ hace más eficiente al minimax, pero es completo y correcto como ya lo era éste, es decir, encuentra la solución óptima para la profundidad dada, si ésta existe, y en el caso de que haya una única meta, es **la misma** que devuelve minimax.
- $\alpha - \beta$ es más eficiente que minimax **en el caso promedio**. Si recorremos los descendientes de un nodo n, sean $n_1, .., n_l$ tal como aparecen en el árbol (del 1 al l) y esa ordenación es arbitraria (es decir, no tiene ninguna relación con el orden de $h(n_1), ..., h(n_l)$), algunas ramas se podarán. Pero es siempre posible encontrar un orden particular en el que las ramas que podrían podarse quedasen las últimas, y en ese caso $\alpha - \beta$ visitaría exactamente los mismos nodos que minimax (pero no más, lo que explica que en la práctica nunca se use minimax).

Ejemplo de minimax y $\alpha - \beta$

Encontrar el camino óptimo y el movimiento preferible usando tanto minimax como $\alpha - \beta$ en este árbol



Arboles para j. parcialmente aleatorios

Se dice que un juego es **parcialmente aleatorio** si el resultado, además de depender de la secuencia de jugadas voluntariamente decididas por cada jugador, depende también de un elemento aleatorio (p. ej.: el lanzamiento de un dado) que restringe las acciones que se pueden tomar. Ejemplos típicos son el parchis, la oca, el backgammon,...

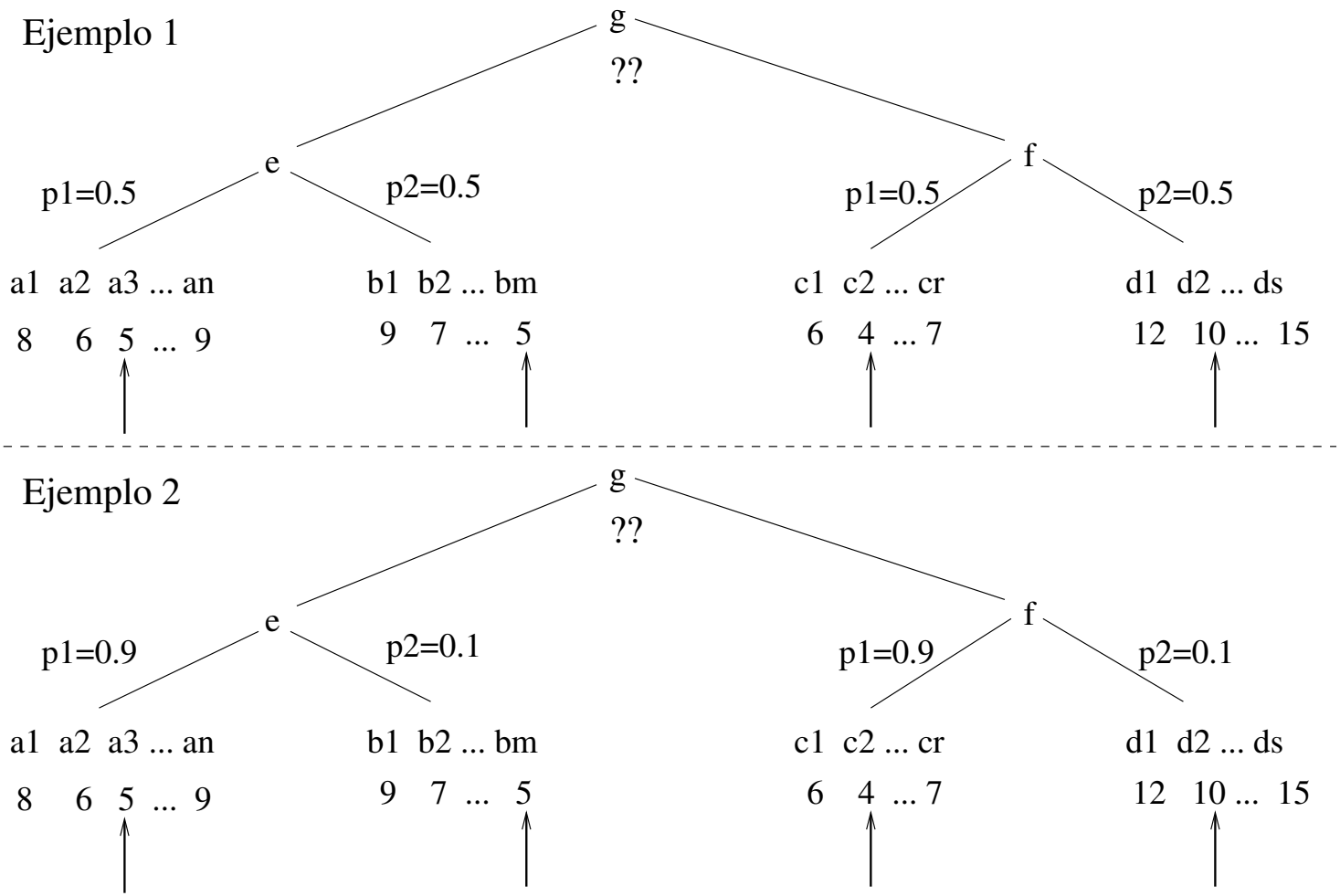
Los árboles en este caso se construyen de manera similar: cada nodo mazimizante n_M tiene una heurística determinista, $h_D(n_M)$ y un conjunto de hijos completamente conocido, sean $n_{m1}, .., n_{mk}$, que serán nodos minimizantes.
Ahora, cada hijo de un nodo minimizante n_m no es un movimiento, sino un resultado posible del suceso aleatorio (de la tirada del contrario), sea $n_{mR1}, \ldots, n_{mRq}$.
Cada resultado sucede con probabilidad p_i y, de suceder, el contrario ejecuta el movimiento que le sea más ventajoso, y que le lleva a un nodo $l_{i,best}$ cuya heurística vale $h_D(l_{i,best})$. Entonces, calculamos la heurística del nodo minimizantes n_{ms} como

$$h_A(n_{ms}) = \sum_{i=1}^{q} p_i h_D(l_{i,best})$$

Hecho esto, se puede usar el algoritmo $\alpha - \beta$ con $h = h_D$ para nodos maximizantes y $h = h_A$ para nodos minimizantes.

Ej. de árbol de j. parcialmente aleatorio

En cada uno de los ejemplos, ¿qué debería hacer el jugador en el estado 'g': moverse hacia el estado 'e' o hacia el 'f'?
(Las flechas marcan el estado minimizante de cada posible resultado aleatorio)



Índice del tema 3: comienzo de la sección 3

1 Formalismo

2 Juegos posicionales: árboles de juego
2.1 Modelos de árboles
2.2 Algoritmo Minimax
2.3 Algoritmo $\alpha - \beta$
2.4 Arboles para juegos parcialmente aleatorios

3 Juegos no posicionales
3.1 Juegos de suma nula
3.2 Juegos de suma no nula

Juegos de suma nula (I)

Juegos de dos jugadores no posicionales de suma nula

Cada uno de los dos jugadores, digamos A y B, puede elegir una acción determinada de entre sus acciones permitidas, sean $\{A_1, .., A_n\}$ y $\{B_1, .., B_m\}$ (no es obligatorio que $n = m$), sin conocer la que está eligiendo el otro jugador. Las acciones se ejecutan y el juego termina inmediatamente.

A la ganancia o pérdida que sufre un jugador la llamaremos **utilidad**, cuyo signo representará si es un cosa u otra.
La utilidad para un jugador, digamos A, viene dada por una matriz en la que se representan en filas sus posibles acciones, y en columnas las del contrario. El elemento a_{ij} es, por tanto, la utilidad para A de ejecutar la acción i si B ha ejecutado la acción j. Para B existe una matriz análoga, con elementos b_{ij}

Decimos que el juego es **de suma nula** si $\forall i, j, a_{ij} = -b_{ij}$, o sea, $\forall i, j, a_{ij} + b_{ij} = 0$. (es decir: si en cualquier resultado, lo que gana un jugador es igual a lo que pierde el otro).

Esto **no necesariamente** significa que juegos con ganancias o pérdidas globales no puedan ser de suma nula; lo serán si $\forall i, j, a_{ij} + b_{ij} = K$, con K constante. Basta redefinir $\hat{a}_{ij} = a_{ij} - \frac{K}{2}; \hat{b}_{ij} = b_{ij} - \frac{K}{2}$

Juegos de suma nula (II)

Un ejemplo de estos juegos es piedra/papel/tijera. La matriz de utilidades del juego, desde el punto de vista del jugador A, es:

	B-piedra	B-papel	B-tijera
A-piedra	0	-1	1
A-papel	1	0	-1
A-tijera	-1	1	0

Desde el punto de vista de B la matriz es la misma, pero traspuesta y con signos opuestos. ¿Qué debería jugar A, si no sabe qué va a hacer B?.
Primera estrategia: **maximin**:
Buscar el mínimo de cada fila, y luego escoger la fila con mayor valor (el máximo) de estos mínimos.
La idea es jugar de modo que lo peor que puede ocurrirnos sea lo mejor (o lo menos malo) posible.
Es la estrategia más conservadora.
En este juego los mínimos por fila son respectivamente -1,-1 y -1, de modo que cualquier elección es igualmente mala.

Juegos de suma nula (III)

Sin embargo, en un juego con una tabla como

	B1	B2	B3
A1	-2	2	-1
A2	2	1	1
A3	3	-3	1

A debería jugar su opción 2, que garantiza que obtendrá una ganancia de al menos 1, y puede ser que más.

¿Qué tendría que jugar B?. Deberemos repetir el procedimiento con su tabla, que sería la de A, pero traspuesta y con signos cambiados.

También podemos usar la misma tabla de A, cambiando en la regla 'filas' por 'columnas' y 'mínimo' por 'máximo', es decir, B debería encontrar el máximo de cada columna, y entonces escoger aquella que de el mínimo de estos máximos, pero preferimos escribir una nueva tabla, para no crear confusión.

Juegos de suma nula (IV)

Resumiendo: las tablas quedan: (la segunda es la primera, traspuesta y cambiada de signo)

Tabla para A

	B1	B2	B3	
A1	-2	2	-1	min=-2
A2	2	(1)	(1)	min=1
A3	3	-3	1	min=-3
				max=1

Tabla para B

	A1	A2	A3	
B1	2	-2	-3	min=-3
B2	-2	-1	3	min=-2
B3	1	(-1)	(-1)	min=-1
				max=-1

Al jugar A su opción A2 obtiene una utilidad (aquí, ganancia) segura de 1 y al jugar B su opción B3 obtiene una utilidad (en este caso, perdida) de -1. Marcaremos con círculo todas las posibilidades de la opción elegida que también den esa misma utilidad.

El juego no es "equilibrado"(en el sentido de imparcial) pero aun así, existe al menos una casilla, la (A2,B3), que está encerrada en círculo para **ambos** jugadores, y en ella los dos obtienen lo mejor que la estrategia maximin les permite. Si existe(n) una o más jugadas de este tipo se dice que para el juego existe uno (o más) punto(s) de **equilibrio de Nash**.

Juegos de suma nula (V)

Sin embargo, en un juego con unas tablas como

Tabla para A

	B1	B2	B3	
A1	4	-1	-3	min=-3
A2	(-2)	1	3	**min=-2**
A3	0	2	-3	min=-3
				max=-2

Tabla para B

	A1	A2	A3	
B1	-4	2	0	min=-4
B2	1	-1	(-2)	**min=-2**
B3	3	-3	3	min=-3
				max=-2

a A le conviene jugar A2, y a B esta vez B2; las utilidades aseguradas son ahora de -2 para A y de -2 para B. Al ser iguales, el juego sí sería imparcial. Pero esta vez no existe una casilla simultáneamente óptima para ambos: para A es la (A2,B1) y para B es la (A3,B2). En este caso se dice que para este juego **no** existe un punto de equilibrio de Nash.

Juegos de suma nula (VI)

No obstante, incluso para juegos de suma nula como este en los que no se puede alcanzar el equilibrio en una partida un teorema debido a von Neumann prueba que, si se juegan múltiples partidas con A eligiendo al azar, pero con probabilides de cada opción $P_a = \{pA_1, ..., pA_n\}$ y con B eligiendo igualmente al azar, con probabilidades $P_b = \{pB_1, ..., pB_m\}$, es posible encontrar ciertos valores para P_a y P_b que constituyen una estrategia de equilibrio global. Es decir, al cabo de suficientes partidas, la ganancia de ambos jugadores será la mejor que cada uno puede alcanzar si el otro también la alcanza.

Para el caso de piedra/papel/tijera no había estrategia de equilibrio, pero si se juegan muchas partidas con cada jugador eligiendo su jugada al azar (con probabilidad $\frac{1}{3}$ para cada una), el número de victorias por jugador tiende a igualarse (cada uno vence aproximadamente la mitad de las veces).

Juegos de suma nula (VII)

Juegos posicionales de dos jugadores de suma nula, resumen:

- Ambos jugadores deberían usar la estrategia maximin, que le garantiza utilidades respectivas de $\alpha = max_i(min_j(a_{ij}))$ y $\beta = max_i(min_j(b_{ij}))$ (siendo $b_{ij} = -a_{ji}$, suma nula).
- Si existen una o más jugadas (i, j) tales que $\alpha = a_{ij}$ y $\beta = b_{ji}$ el juego tiene estrategia de equilibrio (uno o más puntos de equilibrio de Nash). Si cualquiera de los dos jugadores la abandona se arriesga a ganar menos (**no necesariamente** en **esa** jugada, aunque lo más probable es que así ocurra, y por tanto jugar sistemáticamente abandonando el equilibrio le llevará a perder más (o a ganar menos)).
- Aun si no existe ningún punto de equlibrio de Nash, existe una estrategia para la elección en múltiples partidas consistente en las estrategias 'A elige con probabilidades P_a' y 'B elige con probabilidades Q_b' de modo que las ganancias promedio de A y B serían

  $$H_A(P_a, Q_b) = \sum_{i=1}^{m} \sum_{j=1}^{n} p_i q_j a_{ij} \quad \text{y} \quad H_B(P_a, Q_b) = -H_A(P_a, Q_b)$$

- Se dice que esta estrategia es estable porque, para cualesquiera otras estrategias combinadas, P y Q

  $$H_A(P, Q_b) \leq H_A(P_a, Q_b) \leq H_A(P_a, Q) \quad \text{y} \quad H_B(P, Q_b) \geq H_B(P_a, Q_b) \geq H_B(P_a, Q)$$

  lo que significa que, si yo soy un agente racional **y se que mi oponente también lo es**, no abandonaré esta estrategia (porque se que mi oponente tampoco lo hará, de modo que si yo lo hago, a la larga perderé).

Juegos de suma no nula (I)

Juegos de dos jugadores no posicionales de suma no nula

Cada uno de los dos jugadores, digamos A y B, puede elegir una acción determinada de entre sus acciones permitidas, sean $\{A_1, .., A_n\}$ y $\{B_1, .., B_m\}$ (no es obligatorio que $n = m$), sin conocer la que está eligiendo el otro jugador. Las acciones se ejecutan y el juego termina inmediatamente.

A la ganancia o pérdida que sufre un jugador la llamaremos **utilidad**, cuyo signo representará si es un cosa u otra.
La utilidad para un jugador, digamos A, viene dada por una matriz en la que se representan en filas sus posibles acciones, y en columnas las del contrario. El elemento a_{ij} es, por tanto, la utilidad para A de ejecutar la acción i si B ha ejecutado la acción j. Para B existe una matriz análoga, con elementos b_{ij}

Decimos que el juego es **de suma no nula** si $\exists(i,j)/a_{ij} \neq -b_{ji}$ (es decir: si al menos en uno de los posibles resultados, lo que gana un jugador es distinto a lo que pierde el otro. Hay dos matrices, A y B, con $A \neq -B^T$).

Los juegos con ganancias o pérdidas globales desde luego pueden ser de suma no nula; lo serán si $a_{ij} + b_{ji} = K_{ij}$, con K_{ij} un valor no constante.

Juegos de suma no nula (II)

Ejemplo de juego bimatricial: el dilema del prisionero

Sean dos individuos en prisión preventiva por sospecha de asesinato, ambos incomunicados. Sin pruebas materiales suficientes, la condena dependerá de sus testimonios.

- Si ambos se niegan a testificar (callan) el castigo esperado es sólo la prisión preventiva. A la larga se les tendrá que poner en libertad. Demos a esto utilidad -1 para cada prisionero.
- Si ambos confiesan el crimen la condena será leve, y la misma para ambos. Demos a esto utilidad -6 para cada uno.
- Si uno de ellos confiesa, acusando al otro y exculpándose a sí mismo, y el otro calla, el que confiesa queda libre (utilidad 0 para él) y el otro es condenado más severamente (utilidad -9).

Tabla para A	B calla	B confiesa
A calla	-1	-9
A confiesa	0	-6

Tabla para B	A calla	A confiesa
B calla	-1	-9
B confiesa	0	-6

Juegos de suma no nula (III)

Al igual que en los juegos de suma nula, en los de suma no nula se puede aplicar la estrategia maximin (y en este caso es la misma estrategia para ambos jugadores, mirando cada uno su propia tabla).

Si la decisión óptima para A se produce para una decisión de B que también es óptima para él se dice que estamos en un **equilibrio de Nash**. Al igual que en juegos de suma nula, pueden existir juegos en los que se pueda alcanzar ese equilibrio jugando una sóla vez y otros en los que no.

Para el prisionero se puede comprobar que existe equilibrio, y se da para la estrategia en la que A confiesa y B también. Además, en este caso la utilidad es la misma para los dos (-6), pero en general no tiene por qué ser así.

Juegos de suma no nula (IV)

Para juegos de suma no nula en los que no se puede alcanzar el equilibrio en una partida un teorema, este vez de J. Nash, prueba que, si se juegan múltiples partidas con A eligiendo al azar, pero con probabilides de cada opción $P_a = \{p_1, ..., p_n\}$ y con B eligiendo igualmente al azar, con probabilidades $Q_b = \{q_1, ..., q_m\}$, es posible encontrar ciertos valores para P_a y Q_b que constituyen una estrategia de equilibrio global. Es decir, al cabo de suficientes partidas, la ganancia de ambos jugadores será la mejor que cada uno puede alcanzar si el otro también la alcanza.
De hecho, las ganancias respectivas en esta estrategia son:

$$H_A(P_a, Q_b) = \sum_{i=1}^{n}\sum_{j=1}^{m} p_i q_j a_{ij} \quad \text{y} \quad H_B(P_a, Q_b) = \sum_{i=1}^{n}\sum_{j=1}^{m} p_i q_j b_{ij}$$

Se dice que esta estrategia es estable porque, para cualesquiera otras estrategias combinadas, P y Q

$$H_A(P, Q_b) \leq H_A(P_a, Q_b) \quad \text{y} \quad H_B(P_a, Q) \leq H_B(P_a, Q_b)$$

Para encontrar este estrategia se utilizan técnicas de programación lineal.

Juegos de suma no nula (V)

Criterio de optimalidad de Pareto

Además del equilibrio de Nash, existe otro criterio de optimalidad, usable **solo** para juegos de suma **no nula**. Se basa en encontrar la solución que más satisface a ambos jugadores simultáneamente. Para ello se ve si es posible que los dos se muevan a un punto del juego en que **ambas** ganancias aumenten.

Ese punto se determina gráficamente. Dibujamos un diagrama con un punto por cada posibilidad de realización del juego, en las coordenadas dadas por la ganancia obtenida por el primer jugador, y la obtenida por el segundo. Entonces dibujamos el polígono que sea la envolvente convexa de todos estos puntos. Se llama **conjunto de Pareto** al conjunto de vértices del polígono situados más a la derecha o más arriba (explicar la regla del cuadrante).

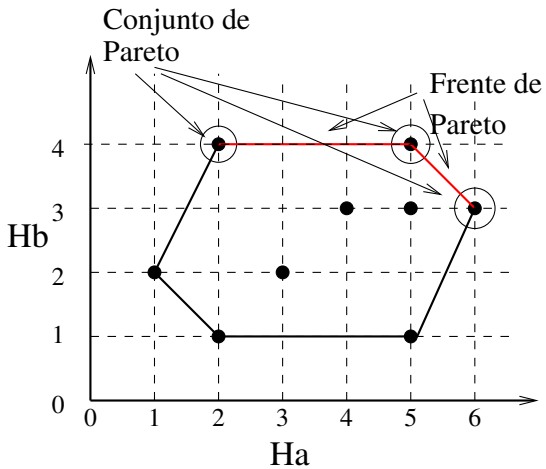


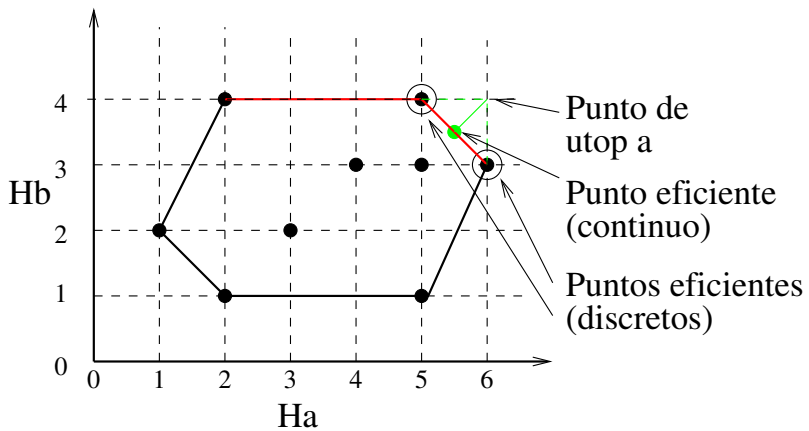
Tabla para A	B1	B2	B3
A1	2	3	4
A2	5	5	6
A3	1	2	5

Tabla para B	B1	B2	B3
A1	1	2	3
A2	3	4	3
A3	2	4	1

(Esta vez la tabla para B se ha escrito traspuesta, por comodidad)

Juegos de suma no nula (VI)

Lógicamente, ambos jugadores podrían decidir moverse a un punto del conjunto de Pareto, pero ¿cuál es mejor?.
Sería el llamado **punto eficiente** que sería el punto del conjunto de Pareto más cercano al llamado **punto de utopía**, que se define como el que daría máxima ganancia tanto para A como para B. Normalmente, el punto de utopía no pertenece al conjunto admisible.



¿Qué criterio sería mejor aplicar, el equlibrio de Nash o el de Pareto?
Para el dilema del prisionero, el equilibrio de Nash es que ambos confiesen, lo que da una utilidad de -6 para ambos. Si embargo, el punto eficiente del conjunto de Pareto es que ambos callen, lo que garantiza una utilidad de -1 para ambos.

Juegos de suma no nula (VII)

Las diferencias entre los dos criterios de optimización son:

- En el punto de equilibrio de Nash ningún jugador puede aumentar **unilateralmente** su ganancia. Si lo hace, a largo plazo acabará perdiendo.
- En cualquier punto del frente de Pareto (incluído el punto eficiente) no es posible aumentar **simultáneamente** las ganancias de ambos.
- En general, el equlibrio de Pareto da mejores resultados en conjunto, pero en la práctica no se aplica porque **no es estable**: si algún jugador lo abandona, puede aumentar su beneficio, tanto a costa del adversario como del bien común. Pero eso no suele ser importante a nivel del propio jugador, de modo que efectivamente lo abandona, obligando a su adversario a abandonarlo también.